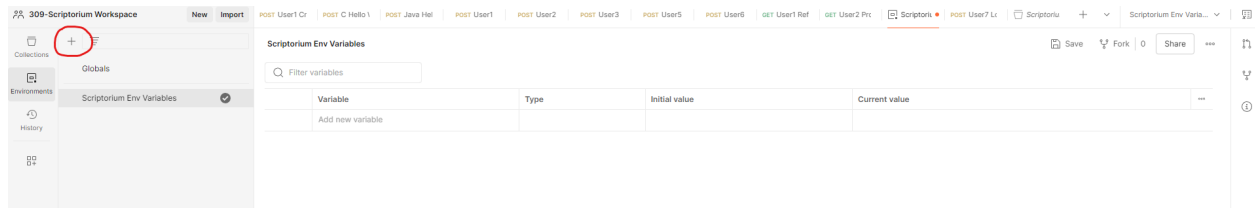


# CSC309: Scriptorium Backend API Documentation

## Postman Guide (Important )

- ❖ For the Postman collection to be functional, an environment must be created for automatic variable storing and updates



## Endpoint Documentation

### Sign up

Endpoint: </api/signup>

Method: POST

Description: Create a new user account

#### Request Body:

- ❖ Email\* [String]: Email for registration
- ❖ FirstName\* [basString]: User's first name
- ❖ Password\* [String]: User's password, must be longer than 8 characters, required
- ❖ PasswordConfirm\* [String]: Repeated password
- ❖ Phone [String]: Phone number

#### Responses:

- ❖ **200 OK**: User signup is successful and a user object is returned.
- ❖ **400 Bad Request**: There were errors in the input such as missing fields or already used emails.

#### Example Request Body:

```
{
  "email": "user@example.com",
  "firstName": "John",
  "lastName": "Doe",
  "password": "password123",
  "passwordConfirm": "password123",
  "phone": "1234567890"
}
```

### Example Response (201 Created)

```
{
  "message": "User created successfully",
  "user": {
    "id": 1,
    "email": "user@example.com",
    "firstName": "John",
    "lastName": "Doe",
    "userType": "USER",
    "avatarUrl": "public/avatars/default.png",
    "phoneNum": "1234567890",
    "createdAt": "2023-10-01T00:00:00.000Z",
    "updatedAt": "2023-10-01T00:00:00.000Z"
  }
}
```

### Example Response (400 Bad request)

```
{
  "error": "Missing required fields: email, password, firstName, and lastName are required"
}
```

---

## Login

Endpoint: `/api/login`

Method: POST

Description: Authenticates a user and returns access and refresh tokens.

Request Body:

- ❖ Email [String]: User's email, required
- ❖ Password [String]: User's password, required

Responses:

- ❖ **200 OK**: User login is successful and the user ID, email, and JWT tokens are returned.
- ❖ **400 Bad Request**: There were errors in the input such as missing fields or invalid credentials.
- ❖ **404 Not Found**: The email could is not linked to a user.

Example Request Body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

#### Example Response (200 OK)

```
{
  "message": "Verified",
  "email": "user@example.com",
  "accessToken": "access_token",
  "refreshToken": "refresh_token"
}
```

#### Example Response (400 Bad Request)

```
{
  "message": "Please provide an email and password"
}
```

#### Example Response (404 Not Found)

```
{
  "message": "Incorrect email or password"
}
```

---

## Profile

Endpoint: `/api/profile`

Method: GET

Description: Retrieves the profile information of the authenticated user.

Request Header: Contains the JWT access token

Response: Returns the User's first name, last name, email, phone number, and avatarUrl

#### Example Request Header:

```
{
  "Authorization": "Bearer access_token"
}
```

#### Example Response (200 OK)

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "user@example.com",
  "phoneNum": "1234567890",
  "avatarUrl": "public/avatars/default.png"
}
```

### Example Response (401 Unauthorized)

```
{
  "message": "Unauthorized"
}
```

---

## Edit Profile

Endpoint: `/api/editProfile`

Method: PUT

Description: Updates the profile information of the authenticated user.

Request Header: Contains a JWT access token.

Example Request Header:

```
{
  "Authorization": "Bearer access_token"
}
```

Example Request Body:

```
{
  "firstName": "Jane",
  "lastName": "Doe",
  "phoneNum": "0987654321",
  "avatarUrl": "public/avatars/new_avatar.png"
}
```

### Example Response (200 OK)

```
{
  "message": "User updated successfully",
  "user": {
    "id": 1,
    "email": "user@example.com",
    "firstName": "Jane",
    "lastName": "Doe",
    "userType": "USER",
    "avatarUrl": "public/avatars/new_avatar.png",
    "phoneNum": "0987654321",
    "createdAt": "2023-10-01T00:00:00.000Z",
    "updatedAt": "2023-10-01T00:00:00.000Z"
  }
}
```

*Example Response (401 Unauthorized)*

```
{  
  "message": "Unauthorized"  
}
```

---

## Refresh Token

Endpoint: `/api/refresh`

Method: GET

Description: Refreshes the access token using the refresh token.

Request Header: Contains a JWT access token.

Example Request Header:

```
{  
  "Authorization": "Bearer refresh_token"  
}
```

*Example Response (200 OK)*

```
{  
  "message": "Access Token Refreshed",  
  "accessToken": "new_access_token"  
}
```

*Example Response (401 Unauthorized)*

```
{  
  "message": "Refresh Token Required"  
}
```

---

## Code Execution

Endpoint: `/api/compile`

Method: POST

Description: Compiles and executes code in python, java, javascript, C, C++

Request Body:

- ❖ Language [String]: The language of the code, optional
- ❖ Code [String]: The code to be compiled, optional
- ❖ Stdin [String]: User's standard input, optional

*Example Request Body:*

```
{
  "language": "python",
  "code": "print('Hello, World!')",
  "stdin": ""
}
```

*Example Response (200 OK)*

```
{
  "output": "Hello, World!\n"
}
```

*Example Response (400 Bad Request)*

```
{
  "message": "Code could not compile check for errors.",
  "error": "Error message"
}
```

---

## Create Blog post

Endpoint: `api/blog-posts`

Method: POST

Description: Creates a new blog post.

Payload:

- ❖ title\* [String]: The title of the new blog post
- ❖ description [String]: A description for the new blog post
- ❖ tags [array of Strings]: Tags to assign to the new blog post

*Example Request Body*

```
{
  "title": "Hello World!",
  "description": "My first Post!",
  "tags": ["new", "hello"]
}
```

*Responses:*

201 Created: Blog post was created. Response contains information about blog post

**400 Bad Request:** The request contained malformed data in the path or body.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

### Example Response (200 OK)

```
{
  "id": 5,
  "createdAt": "2024-11-03T21:14:46.605Z",
  "userId": 2,
  "title": "Hello World!",
  "content": "",
  "description": "My first Post!",
  "isHidden": false,
  "upvotes": 0,
  "downvotes": 0,
  "tags": [
    {
      "label": "new"
    },
    {
      "label": "hello"
    }
  ]
}
```

---

## Get Multiple Blog Posts

Endpoint: `api/blog-posts`

Method: GET

Description: Creates a new blog post.

### Query Parameters:

- ❖ title [String]: The title of the new blog post
- ❖ content [String]: A description for the new blog post
- ❖ tags [String (*comma-separated list of Strings*)]: List of tag names to match to blog posts
- ❖ templates [String (*comma-separated list of Integers*)]: List of template IDs to
- ❖ page [Integer]: Page number of objects. Default is 1
- ❖ limit [Integer]: Number of items in a page. Default is 10
- ❖ sortBy [String]: Sorting order of blog posts. Accepts values "mostValued" and "mostControversial". If none specified, returns blog posts in descending order of creation time.

## Responses:

**200 OK:** Response JSON contains an array of blog posts filtered and sorted by specified parameters.

**400 Bad Request:** The request contained malformed data in the path or query parameters.

### Example Response (200 OK)

```
1  [
2    {
3      "id": 4,
4      "createdAt": "2024-11-03T21:13:41.460Z",
5      "userId": 2,
6      "title": "Hello World!",
7      "content": "",
8      "description": "My first Post!",
9      "isHidden": false,
10     "upvotes": 0,
11     "downvotes": 0,
12     "tags": [
13       {
14         "label": "new"
15       },
16       {
17         "label": "hello"
18       }
19     ]
20   }
21 ]
```

---

## Update Blog post

**Endpoint:** `api/blog-posts/[postId]`

**Method:** PUT

**Description:** Edits a blog post with ID {blogID}.

### Payload:

- ❖ title [String]: The new title of the blog post
- ❖ description [String]: A new description for the new blog post
- ❖ content [String]: Updated content of the blog post
- ❖ tags [array of Strings]: list of tags assign to the blog post
- ❖ Templates [array of Integers]: Integer IDs linked to the blog post



### Example Request Body

```
{
  "title": "Hello World!",
  "description": "My first Post!",
  "tags": ["new", "hello"]
}
```

### Responses:

200 OK: Blog post was updated. Response contains information about blog post

**400 Bad Request:** The request contained malformed data in the path or body.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**404 Not Found:** Could not find a post with {blogId}. This could also mean that the blog post you are trying to delete does not belong to the user deleting it.

### Example Response (200 OK)

```
{
  "id": 5,
  "createdAt": "2024-11-03T21:14:46.605Z",
  "userId": 2,
  "title": "Hello World!",
  "content": "",
  "description": "My first Post!",
  "isHidden": false,
  "upvotes": 0,
  "downvotes": 0,
  "tags": [
    {
      "label": "new"
    },
    {
      "label": "hello"
    }
  ]
}
```

### Example Response (401 Unauthorized)

```
{
  "message": "Unauthorized"
}
```

---

## Delete Blog post

**Endpoint:** `api/blog-posts/[postId]`

**Method:** DELETE

**Description:** Deletes a blog post with ID {blogId}.

**Responses:**

**200 OK:** Blog post was deleted. Response contains success message

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**404 Not Found:** Could not find a post with {blogId}. This could also mean that the blog post you are trying to delete does not belong to the user deleting it.

**Example Response (200 OK)**

```
1  {
2    |    "message": "Successfully deleted Hello World!."
3  }
```

**Example Response (401 Unauthorized)**

```
{
  "message": "Unauthorized"
}
```

---

## Get Multiple Comments

**Endpoint:** `/api/blog-posts/[postId]/comments`

**Method:** GET

**Description:** Creates a new comment on a blog post with {postId}

**Query Parameters:**

- ❖ `page [Integer]`: Page number of comments. Default is 1
- ❖ `limit [Integer]`: Number of comments in a page. Default is 10
- ❖ `sortBy [String]`: Sorting order of comments. Accepts values "mostValued" and "mostControversial". If none specified, returns comments in descending order of creation time.

**Responses:**

**200 OK:** Response JSON contains an array of comments filtered and sorted by specified parameters.

**400 Bad Request:** The request contained malformed data in the path or query parameters.

**Example Request Body:**

```
{
  "content": "This is a comment.",
  "parent": 1
}
```

**Example Response (200 OK)**

```
1  [
2    {
3      "id": 1,
4      "createdAt": "2024-11-03T22:09:24.038Z",
5      "content": "great post!!",
6      "userId": 2,
7      "isHidden": false,
8      "postId": 5,
9      "parentId": null,
10     "upvotes": 1,
11     "downvotes": 0,
12     "replies": [
13       {
14         "id": 2,
15         "createdAt": "2024-11-03T22:14:06.870Z",
16         "content": "I thought so as well!",
17         "userId": 2,
18         "isHidden": false,
19         "postId": 5,
20         "parentId": 1,
21         "upvotes": 0,
22         "downvotes": 0
23       }
24     ]
25   }
26 ]
```

**Example Response (400 Bad Request)**

```
1  {
2    "message": "Invalid blog ID type provided"
3  }
```

---

## Create Comment

**Endpoint:** `/api/blog-posts/{postId}/comments`

**Method:** POST

**Description:** Create a new comment on a specific blog post with ID {postId}

**Responses:**

200 OK: Blog post was deleted. Response contains success message

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**404 Not Found:** Could not find a post with {postId}.

**Example Request Body:**

```
{
  "content": "This is a comment.",
  "parent": 1
}
```

**Example Response (201 Created)**

```
{
  "content": "This is a comment.",
  "createdAt": "2023-10-01T00:00:00.000Z",
  "upvotes": 0,
  "downvotes": 0,
  "post": {
    "connect": {"id": 1}
  },
  "parent": {
    "connect": {"id": 1}
  },
  "createdBy": {
    "id": 1
  },
  "replies": []
}
```

**Example Response (400 Bad Request)**

```
{
  "message": "Blog ID not provided"
}
```

**Example Response (401 Unauthorized)**

```
{
  "message": "Unauthorized"
}
```

---

## Hide Comment

**Endpoint:** `/api/blog-posts/[postId]/comments/[commentId]/hide`

**Method:** PUT

**Description:** Updates the comment with {commentId} to have the isHidden value of true.

**200 OK:** Response JSON contains information about hidden comment

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**403 Forbidden:** Logged in user is not an admin

**404 Not Found:** Could not find a comment with {commentId} or a post with {postId}.

---

## Report Comment

**Endpoint:** /api/blogs-posts/{postId}/comments/commentId/report

**Method:** POST

**Description:** Reports a specific comment.

**Request Payload:**

❖ explanation[String]: explanation of why report was made

**Example Request Body:**

```
{
  "explanation": "This comment is inappropriate."
}
```

**200 OK:** Response JSON contains information about reported comment

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**404 Not Found:** Could not find a comment with {commentId} or a post with postId.

---

## Hide Blog Post

**Endpoint:** /api/blogs-posts/{postId}/comments/commentId/report

**Method:** PUT

**Description:** Updates the blog post with {postId} to have the isHidden value of true.

**200 OK:** Response JSON contains information about hidden comment

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**403 Forbidden:** Logged in user is not an admin

**404 Not Found:** Could not find a post with {postId}.

---

## Report Blog post

**Endpoint:** /api/blogs-posts/{postId}/report

**Method:** POST

**Description:** Reports a specific post with {postId}.

### Request Payload:

- ❖ explanation[String]: explanation of why report was made

### Example Request Body:

```
{  
  "explanation": "This comment is inappropriate."  
}
```

**200 OK:** Response JSON contains information about reported comment

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**404 Not Found:** Could not find a post with {postId}.

---

## Rate Blog Post

**Endpoint:** /api/blogs-posts/{postId}/comments/{commentId}/rate

**Method:** PUT

**Description:** Updates the blog post with {postId} to have the isHidden value of true.

### Request Body:

- ❖ ratingType[String]:

**200 OK:** Response JSON contains information about hidden comment

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**404 Not Found:** Could not find a post with {postId} or a comment with [commentId].

---

## Rate Comment

**Endpoint:** /api/blogs-posts/{postId}/comments/commentId/rate

**Method:** PUT

**Description:** Updates the blog post with {postId} to have the isHidden value of true.

### Request Body:

- ❖ page [Integer]: Page number of comments. Default is 1
- ❖ limit [Integer]: Number of comments in a page. Default is 10

**200 OK:** Response JSON contains information about hidden comment

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**404 Not Found:** Could not find a post with {postId}.

---

## Get Content Sorted by Reports

**Endpoint:** /api/reports

**Method:** GET

**Description:** Updates the blog post with {postId} to have the isHidden value of true.

**Query Parameters:**

- ❖ page [Integer]: Page number of comments. Default is 1
- ❖ limit [Integer]: Number of comments in a page. Default is 10

**200 OK:** Response a JSON array contains information about content, including a report count.

**400 Bad Request:** The request contained malformed data in the path.

**401 Unauthorized:** Bad authorization or expired token. You should refresh if possible or log in again.

**403 Forbidden:** Logged in user is not an admin

---

## Create Template

**Endpoint:** /api/Template

**Method:** POST

**Description:** Creates a new template. Fails if parent template with parentID exists

**Request Body:**

- ❖ title\* [String] : The title/name of the code template
- ❖ explanation\* [String] : A detailed explanation of what the code does
- ❖ language\* [String] : The programming language of the code snippet
- ❖ code\* [String] : The actual code snippet/template content
- ❖ authorID\* [String] : The unique identifier of the user creating the template
- ❖ tags [Array<Str>] : Optional array of tag names to categorize the template
- ❖ parentID [Int] : Optional ID referencing another template this one is forked from. If provided, creates a parent-child relationship between templates.

**Responses:**

- ❖ **201 Created:** Template creation is successful and a Template object is returned. With message

- ❖ **201 Forked**: Template fork is successful and a Template object is returned. With message
- ❖ **400 Bad Request**: There were errors in the input such as missing required fields or no existing author with ID.

#### Example Request Body:

```
{
  "title": "React Button Component",
  "explanation": "A reusable button component with customizable styles",
  "language": "javascript",
  "code": "const Button = ({ text, onClick, variant = 'primary' }) => {\n  return\n    (\n      <button\n        className={`btn btn-${variant}`} \n        onClick={onClick}\n      >\n        {text}\n      </button>\n    );\n};",
  "authorId": 1,
  "tags": ["react", "component", "frontend"]
}
```

#### Example Response (201 Created):

```
{
  "message": "Template successfully created",
  "template": {
    "id": 1,
    "title": "React Button Component",
    "explanation": "A reusable button component with customizable styles",
    "language": "javascript",
    "code": "const Button = ({ text, onClick, variant = 'primary' }) => {\n  return (\n\n    <button\n      className={`btn btn-${variant}`} \n      onClick={onClick}\n    >\n      {text}\n    </button>\n  );\n};",
    "authorId": 1,
    "parentId": null,
    "createdAt": "2024-11-04T00:40:22.336Z",
    "updatedAt": "2024-11-04T00:40:22.336Z",
    "author": {
      "id": 1,
      "firstName": "Admin",
      "lastName": "User",
      "email": "admin@scriptorium.com"
    },
    "parent": null,
    "tags": []
  }
}
```



**Example Response (201 Forked):**

```
{
  "message": "Template successfully forked from \"React Button Component\".",
  "template": {
    "id": 6,
    "title": "React Button Component",
    "explanation": "A reusable button component with customizable styles",
    "language": "javascript",
    "code": "const Button = ({ text, onClick, variant = 'primary' }) => {\n
      return (\n    <button\n      className={`btn btn-${variant}`} \n
      onClick={onClick}\n    >\n      {text}\n    </button>\n  );\n};",
    "authorId": 1,
    "parentId": 4,
    "createdAt": "2024-11-04T01:51:53.642Z",
    "updatedAt": "2024-11-04T01:51:53.642Z",
    "author": {
      "id": 1,
      "firstName": "Admin",
      "lastName": "User",
      "email": "admin@scriptorium.com"
    },
    "parent": {
      "id": 4,
      "title": "React Button Component",
      "author": {
        "id": 1,
        "firstName": "Admin",
        "lastName": "User"
      }
    }
  },
}
```

```

    "tags": [
      {
        "id": 1,
        "name": "react",
        "createdAt": "2024-11-04T00:40:51.384Z",
        "updatedAt": "2024-11-04T00:40:51.384Z"
      },
      {
        "id": 2,
        "name": "component",
        "createdAt": "2024-11-04T00:40:51.384Z",
        "updatedAt": "2024-11-04T00:40:51.384Z"
      },
      {
        "id": 3,
        "name": "frontend",
        "createdAt": "2024-11-04T00:40:51.384Z",
        "updatedAt": "2024-11-04T00:40:51.384Z"
      }
    ]
  }
}

```

*Example Response (400 Bad request):*

```

{
  "error": "Author ID not found"
}

```

---

## Get Templates

Endpoint: [api/Template](#)

Method: GET

Description: Gets Template based on search query.

Query Parameters:

- ❖ id [Int] : Unique identifier for a specific template
- ❖ authorID [Str] : Filter templates by their author/creator
- ❖ language [Str] : Filter templates by programming language
- ❖ tag [Str] : Filter templates by tag name

Responses:

- ❖ **200 Success**: Template search was successful and an array of Template objects are returned. With message
- ❖ **404 Template not Found**: There were no templates found within search query.

Example Request Body:

```
http://localhost:3000/api/Template?id=1
```

Example Response (200 success):

```
[
  {
    "id": 1,
    "title": "React Button Component",
    "explanation": "A reusable button component with customizable styles",
    "language": "javascript",
    "code": "const Button = ({ text, onClick, variant = 'primary' }) => {\n
      return (\n    <button\n      className={`btn btn-${variant}`} \n
      onClick={onClick}\n    >\n      {text}\n    </button>\n  );\n};",
    "authorId": 1,
    "parentId": null,
    "createdAt": "2024-11-04T00:40:22.336Z",
    "updatedAt": "2024-11-04T00:40:22.336Z",
    "author": {
      "id": 1,
      "firstName": "Admin",
      "lastName": "User",
      "email": "admin@scriptorium.com"
    },
    "tags": [],
    "savedBy": []
  }
]
```

Example Response (400 Bad request):

```
{
  "error": "Template not found"
}
```

---

## Update Template

Endpoint: `/api/Template/[id]`

Method: POST

Description: Creates a new template

Request Body:

- ❖ email\* [String] : The user's email address
- ❖ explanation\* [String] : A detailed description of the code template
- ❖ language\* [String] : The programming language of the code snippet
- ❖ code\* [String] : The actual code snippet/template content

- ❖ tags [Array<Str>]: Optional array of descriptive tags for categorization

### Responses:

- ❖ **200 OK**: Template creation is successful and a Template object is returned. With message
- ❖ **400 Bad Request**: There were errors in the input such as missing required fields. Or tags are not in array form
- ❖ **404 Template not found**: Template with id was not found

### Example Request Body:

```
{
  "title": "Updated React Button Component",
  "explanation": "An updated reusable button component with new styles",
  "language": "javascript",
  "code": "const Button = ({ text, onClick, variant = 'primary' }) => {...}",
  "tags": ["react", "component", "frontend"]
}
```

### Example Response (200 OK):

```
{
  "id": 1,
  "title": "Updated React Button Component",
  "explanation": "An updated reusable button component with new styles",
  "language": "javascript",
  "code": "const Button = ({ text, onClick, variant = 'primary' }) => {...}",
  "authorId": 1,
  "parentId": null,
  "createdAt": "2024-11-04T00:40:22.336Z",
  "updatedAt": "2024-11-04T01:32:38.527Z",
  "tags": [
    {
      "id": 1,
      "name": "react",
      "createdAt": "2024-11-04T00:40:51.384Z",
      "updatedAt": "2024-11-04T00:40:51.384Z"
    },
    {
      "id": 2,
      "name": "component",
      "createdAt": "2024-11-04T00:40:51.384Z",
      "updatedAt": "2024-11-04T00:40:51.384Z"
    },
    {
      "id": 3,
      "name": "frontend",
      "createdAt": "2024-11-04T00:40:51.384Z",
      "updatedAt": "2024-11-04T00:40:51.384Z"
    }
  ],
  "author": {
    "id": 1,
    "firstName": "Admin",
    "lastName": "User",
    "email": "admin@scriptorium.com"
  }
}
```

**Example Response (400 Bad request)**

```
{
  "error": "Missing required fields",
  "details": {
    "title": null,
    "explanation": null,
    "language": "Language is required",
    "code": null
  }
}
```

**Example Response (404 Template not found):**

```
{
  "error": "Template not found"
}
```

---

## Delete Template

Endpoint: `/api/Template/[id]`

Method: POST

Description: Delete template with id

Responses:

- ❖ **204 No Content**: Template was successfully deleted
- ❖ **404 Template not found**: Template with id was not found

**Example Request Body:**

```
http://localhost:3000/api/Template/1
```

**Example Response (204 No Content):**

**Example Response (404 Template not found):**

```
{
  "error": "Template not found or delete failed"
}
```

**Example Response (400 Bad request)**

```
{
  "error": "Missing required fields",
  "details": {
    "title": null,
    "explanation": null,
    "language": "Language is required",
    "code": null
  }
}
```

*Example Response* (404 Template not found):

```
{  
  "error": "Template not found"  
}
```